Project CCA

Computational Optimizations of Structured Matrices, Toeplitz case

Ali Arda BARUT
Rafael LLAMAS

April 30, 2026

Contents

1 Introduction

Structured matrices can allow optimizations in certain matrix operations. We will be studying the cases of Toeplitz matrices and its property of diagonally descending elements.

Each section is an implementation topic on which we state our implementation, complexity and the results.

2 Definitions

We will discuss what are Toeplitz Matrices, how are they defined and what are their properties such that we can take advantage during matrix implementations.

2.1 Toeplitz Matrices

A Toeplitz matrix is in which each element in first row and first column descends diagonally. For a matrix $A \in \mathbb{K}^{n\times m}$, it satisfies the condition $A_{i,j} = A_{i+1,j+1}$. Formally, it is defined by a sequence of elements $\{a_k\}$ such that:

$$A = \begin{pmatrix} a_0 & a_{-1} & a_{-2} & \cdots & a_{-(m-1)} \\ a_1 & a_0 & a_{-1} & \cdots & a_{-(m-2)} \\ a_2 & a_1 & a_0 & \cdots & a_{-(m-3)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & a_{n-3} & \cdots & a_{n-m} \end{pmatrix} \tag{1}$$

$$A_{i,j} = a_{i-j}, \quad 0 \le i < n,\ 0 \le j < m$$

2.1.1 Quasi Toeplitz Matrices

A Quasi-Toeplitz matrix is not strictly Toeplitz but shares similar structural properties. We can also define Quasi-Toeplitz matrix as a Toeplitz Matrix that has noise within. Notice that we can still benefit the structural advantages of Toeplitz partially in these matrices.

2.2 Displacement Matrices

For a matrix $A \in \mathbb{K}^{n\times m}$, we define the displacement matrix ∇A using:

$$\nabla A = A - ZAZ^T \tag{2}$$

where Z represents the lower shift matrix and Z^T represents the upper shift matrix. Essentially, Z is a matrix with ones on the first sub-diagonal and zeros everywhere else:

$$Z = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix} Z^T = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix} \tag{3}$$

Multiplying a matrix A by Z from the left (ZA) shifts the rows of A down by one (zeros at the top). Multiplying by Z^T from the right (AZ^T) shifts the columns to the right, (zeros at the first column). Resulting in an operation ZAZ^T that shifts the entire matrix A one step to the bottom-right.

2.2.1 Mathematical Approach

For a generic Toeplitz matrix of size $n \times m$, this subtraction results in zeros everywhere except for the first row and the first column.

$$\nabla A = \begin{pmatrix} y_0 & x_1 & x_2 & \cdots & x_{m-1} \\ y_1 & 0 & 0 & \cdots & 0 \\ y_2 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{n-1} & 0 & 0 & \cdots & 0 \end{pmatrix} \in \mathbb{K}^{n \times m} \tag{4}$$

This allows us to represent the $n \times m$ matrix using only $O(n + m)$ elements rather than storing the full $O(nm)$ dense matrix.

2.2.2 Computational Approach

While mathematical definition involves matrix multiplication, in this case $O(n^2 * m)$ or similar depending on the multiplication method used, calculating ∇A based on matrix multiplication is inefficient.

The displacement calculation can be reduced to an element based loop with complexity $O(nm)$:

$$(\nabla A)_{i,j} = \begin{cases} A_{i,j} & \text{if } i = 0 \text{ or } j = 0 \\ A_{i,j} - A_{i-1,j-1} & \text{otherwise} \end{cases} \tag{5}$$

This approach allows us to iterate from the bottom row to the top updating values in place.

2.3 Generator Matrices

Displacement matrices, ∇A, typically have a low rank α, we call this displacement rank. For Toeplitz matrices, α is generally small ($\alpha \leq 2$). This allows us to store efficiently the displacement matrix into two smaller matrices G and H.

While the product $G \times H^T$ results in the displacement ∇A, the original matrix A can be reconstructed using the columns of the generators mentioned at Proposition 10.5 in [1]:

$$A - ZAZ^T = GH^T \quad \text{iff} \quad A = \sum_{i=1}^{\alpha} L(x_i)U(y_i), \tag{6}$$

where x_i (and thus y_i) are the columns of the generator G (and thus H), where for a column vector $v = [v_0 \dots v_{n-1}]^T$, $L(v)$ denotes the lower triangular Toeplitz matrix:

$$L(v) = \begin{pmatrix} v_0 & 0 & \dots & 0 \\ v_1 & v_0 & \dots & 0 \\ \vdots & \ddots & \dots & \vdots \\ v_{n-1} & \dots & v_1 & v_0 \end{pmatrix}$$

and $U(v)$ denotes the upper triangular Toeplitz matrix $L(v)^T$.

The matrix A can be written as a sum of elements of lower and upper triangular matrices:

$$A = \sum_{k=1}^{\alpha} L(g_k)U(h_k) \tag{7}$$

- $L(g_k)$ is a lower triangular Toeplitz matrix with the first column g_k.
- $U(h_k)$ is an upper triangular Toeplitz matrix with the first row h_k^T.

2.4 Displacement Operators

We define two displacements operators:

$$\begin{aligned} \phi_+(A) &= A - ZAZ^T = \nabla A \\ \phi_-(A) &= A - Z^T AZ \end{aligned} \tag{8}$$

In other words, $\phi_-(A)$ is the substraction of minuend A where subtrahend is the matrix A but with the columns shifted up and rows to right once.

Computationally, it follows the same path as the calculation of ∇A. Main difference being we can run this time in increasing order:

$$\phi_-(A)_{i,j} = \begin{cases} A_{i,j} & \text{if } i = n-1 \text{ or } j = m-1 \\ A_{i,j} - A_{i+1,j+1} & \text{otherwise} \end{cases} \tag{9}$$

Where `n` and `m` are respectfully the number of rows and columns of matrix A.

2.5 Conclusion & Implementation

We conclude the section of the definitions. Implementations of these concepts can be found in:

```
Project
\   src
|   \   displacement_matrices
|   |   |   displacement_matrices.c
|   |   |   displacement_matrices.h
```

gr_mat_displacement(gr_mat_t D, gr_mat_t A, disp_type_t type, gr_ctx_t ctx) Is the function taking as parameter the matrix A and returns into the matrix D, indicating destination, the displacement matrix of A. According to the type of displacement we pass via the parameter `type`, it calculates the implementations (5) or (9).

int gr_mat_G_H(gr_mat_t G, gr_mat_t H, gr_mat_t A, disp_type_t type, gr_ctx_t ctx)) Is the function that computes the generator matrices G and H^T for an input matrix A. Based on the displacement equations discussed previously, the equation (7), the function first calculates the displacement matrix according to the specified type of displacement. Since the displacement of a structured matrix (like a Toeplitz matrix) inherently results in a matrix with a low rank (α), D can be factored into two smaller matrices such that $D = GH^T$. The function achieves this by performing an LU decomposition on the displacement matrix D. It then extracts the independent columns to form G and the corresponding rows to form H^T. This is where the storage economy for structured matrices come from.

int gr_mat_reconstruct_A(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx)

3 Operation

3.1 Addition

This code can be found in:

```
Project
\   src
|   \   operations
|   |   |   addition.c
|   |   |   addition.h
```

3.1.1 Introduction

Grace au générateurs de déplacement, nous pouvons addtionner nos matrices sans utiliser la formule standart. En effet, selon la propriété 10.7 et 10.8 du cours (ref cours ICI), on apprends que la somme de 2 matrices toeplitz peut etre simplfié par la concatenation de leurs générateurs de déplacement. Cela permet de passer a un coup de O($n \times m$) a un cout de O($\alpha \times m$) pour 2 matrices A et B de tailles $n \times m$ et leurs générateurs de déplacement de taille α pour ceux de A et β pour ceux de B. Pour faire la concatenation avec flint on peut utiliser la fonction suivante :

```
int gr_mat_concat_horizontal(gr_mat_t RES,gr_mat_t A,gr_mat_t B,gr_ctx_t ctx);
```

3.1.2 Exemple

On va prendre les matrices A, B et C le résultat de l'addition de A et B.Toutes les matrices sont dans $Z/_{13}Z$ pour cette exemple.

$$A = \begin{pmatrix} 0 & 7 & 10 \\ 5 & 0 & 7 \\ 0 & 5 & 0 \end{pmatrix}, B = \begin{pmatrix} 9 & 12 & 4 \\ 0 & 9 & 12 \\ 3 & 0 & 9 \end{pmatrix}, C = \begin{pmatrix} 9 & 6 & 1 \\ 5 & 9 & 6 \\ 3 & 5 & 9 \end{pmatrix} \tag{10}$$

si on prends les générateurs de déplacement de A = (T,U) et B = (G,H)

$$T = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, U = \begin{pmatrix} 5 & 0 \\ 0 & 7 \\ 0 & 10 \end{pmatrix} \tag{11}$$

$$G = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 9 & 1 \end{pmatrix}, H = \begin{pmatrix} 9 & 0 \\ 12 & 9 \\ 4 & 3 \end{pmatrix} \tag{12}$$

et que l'on fait la concaténation on obtiens

$$RES_G = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 9 & 1 \end{pmatrix} \tag{13}$$

$$RES_H = \begin{pmatrix} 5 & 0 & 9 & 0 \\ 0 & 7 & 12 & 9 \\ 0 & 10 & 4 & 3 \end{pmatrix} \tag{14}$$

Dans notre projet, il suffira d'appeler la fonction:

```
int gr_mat_addition_generateur(gr_mat_t G_a, gr_mat_t H_a, gr_mat_t G_b, gr_mat_t H_b,
                               gr_mat_t G_c, gr_mat_t H_c,gr_ctx_t ctx);
```

avec les bons parametres pour obtenir les générateurs de déplacement G et H de D.

pour verifier notre résultat on peut utiliser la fonction

```
int gr_mat_reconstruct_A(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx);
```

pour recomposer la matrice D que l'on a obtenue :

$$D = \begin{pmatrix} 9 & 6 & 1 \\ 5 & 9 & 6 \\ 3 & 5 & 9 \end{pmatrix} \tag{15}$$

On remarque bien que celle-ci est identique a la matrices C et donc que pour faire l'addition de 2 matrice toeplitz, on peut utiliser seulement leurs générateurs de déplacement.

3.1.3 Benchmark

Le benchmark obtenu met en évidence les résultats suivants:

```
=== BENCHMARK ADDITION (10 runs) ===
Size 128x128 :
 Operation    | Average (ms) | Median (ms)
--------------|--------------|--------------
 Generators   |  3.577e-03   |  3.422e-03
 Flint Dense  |  1.373e-02   |  1.208e-02
Size 1024x1024 :
 Operation    | Average (ms) | Median (ms)
--------------|--------------|--------------
 Generators   |  3.205e-02   |  3.365e-02
 Flint Dense  |  2.520e+00   |  3.018e+00
Size 4096x4096 :
 Operation    | Average (ms) | Median (ms)
--------------|--------------|--------------
 Generators   |  1.148e-01   |  1.104e-01
 Flint Dense  |  2.918e+01   |  2.915e+01
```

-Pour les petites matrices (128x128): notre fonction d'addition avec les générateurs de déplacement est plus rapide que la fonction naive de flint (3.577e-03 ms vs 1.373e-02 ms)

-Pour les matrices de tailles 1024x1024: notre fonction d'addition avec les générateurs de déplacement est plus rapide que la fonction naive de flint (3.205e-02 ms vs 2.520 ms)

-Pour de grandes matrices (4096x4096): notre fonction d'addition avec les générateurs de déplacement est plus rapide que la fonction naive de flint (1.148e-01 ms vs 2.918e+01 ms)

(ici courbe des temps) blablabla...explication complexité...blablabla

3.2 Multiplication

This code can be found in:

```
Project
\   src
|   \   operations
|   |   |   multiplication.c
|   |   |   multiplication.h
```

3.2.1 Introduction

Toujours avec les générateurs de déplacement, on peut simplifier la multiplication de 2 matrices et donc réduire son cout en concatenant :

- les générateurs de déplacement de A=(T,U) et B=(G,H)
- $W = Z \times A \times Z^t \times G$
- $V = B^t \times U$
- a (resp. b) un vecteur qui est la dernière colonne de$Z \times A$ (resp. $Z \times B^t$)

Cela nous permet de passer de $O(n^3)$ pour l'algo naif a $O(\alpha^2 \times M(n))$ pour la version que nous implémentons. Ainsi nous obtenons W de la façon suivante :

```
    error = gr_mat_apply_Zt(Tmp, G_b, ctx);
error = gr_mat_mul_vector(W, G_a, H_a, Tmp, ctx);
error = gr_mat_apply_Z(W, W, ctx);
```

Par ailleurs, j'obtiens V par l'opération ci-dessous:

```
    error = gr_mat_mul_vector(V, H_b, G_b, H_a, ctx);
```

etant precise que pour avoir B^t je fais le produit inverse qui est $H \times G$ et non $G \times H$

Pour récupérer la dernière colonne de $Z \times A$ et $Z \times B^t$ nous choisissons de proceder ainsi qu'il suit

```
slong m_a = gr_mat_nrows(H_a, ctx);
gr_mat_init(LastCol_m, m_a, 1, ctx);
error = gr_mat_zero(LastCol_m, ctx);
error = gr_one(gr_mat_entry_ptr(LastCol_m, m_a - 1, 0, ctx), ctx);
error = gr_mat_mul_vector(a, G_a, H_a, LastCol_m, ctx);
error = gr_mat_apply_Z(a, a, ctx);

slong m_bt = gr_mat_nrows(G_b, ctx);
gr_mat_init(LastCol_k, m_bt, 1, ctx);
error = gr_mat_zero(LastCol_k, ctx);
error = gr_one(gr_mat_entry_ptr(LastCol_k, m_bt - 1, 0, ctx), ctx);
error = gr_mat_mul_vector(b, H_b, G_b, LastCol_k, ctx);
error = gr_mat_apply_Z(b, b, ctx);
```

Nous utilisons le vecteur

$$Vecteur = \begin{pmatrix} 0 \\ 0 \\ . \\ . \\ . \\ 0 \\ 1 \end{pmatrix} \tag{16}$$

afin de récupérer la dernière colonne de la matrice, ce vecteur est constitué que de 0 à l'exclusion du dernier élément de la colonne dont la valeur est fixé a 1.

3.2.2 Multiplication Vecteur

Pour faire notre multiplication de matrice via leurs générateurs de déplacement, nous avons besoin d'avoir une fonction qui va multiplier un vecteur avec des générateurs de déplacement pour éviter de recomposer les matrices.

Tout d'abord, on initialise notre résultat à 0

```
 error = gr_mat_zero(Res, ctx);
```

Nous partons du principe selon lequelle une matrices peut etre représentée par plusieurs polynome. Ainsi on détermine que chaque colonne de notre vecteur X est un polynome. Chaque ligne de nos matrices correspondent egalement à un polynome. Pour obtenir notre résultat, nous effectuons les opérations dans suivants:

```
for (slong j = 0; j < X_cols; j++) {

  error = gr_poly_reverse(ph, ph, m, ctx);
  error = gr_poly_mul(p_tmp, ph, px, ctx);
  error = gr_poly_shift_right(p_tmp, p_tmp, m - 1, ctx);
  error = gr_poly_mul(p_tmp, pg, p_tmp, ctx);
  }
```

Avec: -X$_c$*olslenombredecolonnedenotrevecteur*$-ph, pgetpxlespolynomedeH, GetX-p_t$*empleproduitintermdiairedephetp*

Pour finir, nous remplissons res avec p$_t$*emplabonneposition.*

3.2.3 Exemple

On va prendre les matrices A, B et C le résultat de la multiplication de A et B.Toutes les matrices sont dans $Z/_{13}Z$ pour cette exemple.

$$A = \begin{pmatrix} 10 & 5 & 8 \\ 5 & 10 & 5 \\ 3 & 5 & 10 \end{pmatrix}, B = \begin{pmatrix} 0 & 3 & 0 \\ 3 & 0 & 3 \\ 0 & 3 & 0 \end{pmatrix}, C = \begin{pmatrix} 2 & 2 & 2 \\ 4 & 4 & 4 \\ 2 & 0 & 2 \end{pmatrix} \tag{17}$$

si on prends les générateurs de A = (T,U) et B = (G,H)

$$T = \begin{pmatrix} 1 & 0 \\ 7 & 1 \\ 12 & 11 \end{pmatrix}, U = \begin{pmatrix} 10 & 0 \\ 5 & 4 \\ 8 & 9 \end{pmatrix} \tag{18}$$

$$G = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, H = \begin{pmatrix} 3 & 0 \\ 0 & 3 \\ 0 & 0 \end{pmatrix} \tag{19}$$

on doit calculer W et V via leurs formule donné au début.

$$W = \begin{pmatrix} 0 & 0 \\ 10 & 0 \\ 5 & 0 \end{pmatrix}, V = \begin{pmatrix} 2 & 12 \\ 2 & 1 \\ 2 & 12 \end{pmatrix} \tag{20}$$

Puis faire la concatenation de T,W et α pour obtenir RES_G et la concatenation de V,H et β pour obtenir RES_H (il faut faire la concatenation dans l'ordre que j'ai cité les éléments).

$$RES_G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 7 & 1 & 10 & 0 & 8 \\ 12 & 11 & 5 & 0 & 5 \end{pmatrix} \tag{21}$$

$$RES_H = \begin{pmatrix} 2 & 12 & 3 & 0 & 0 \\ 2 & 1 & 0 & 3 & 0 \\ 2 & 12 & 0 & 0 & 10 \end{pmatrix} \tag{22}$$

Dans notre projet, il suffira d'appeler la fonction:

```
int gr_mat_mul_generator(gr_mat_t G_c, gr_mat_t H_c, gr_mat_t G_a,
                         gr_mat_t H_a, gr_mat_t G_b, gr_mat_t H_b,
                         gr_ctx_t ctx);
```

avec les bons parametre pour obtenir les générateur de déplacement G et H de D.

pour verifier notre résultat on peut utiliser la fonction

```
int gr_mat_reconstruct_A(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx);
```

pour recomposer la matrice que l'on a obtenue D :

$$D = \begin{pmatrix} 2 & 2 & 2 \\ 4 & 4 & 4 \\ 2 & 0 & 2 \end{pmatrix} \tag{23}$$

La matrice D est bien égale a C donc notre facon de multiplier les 2 matrices est correct dans ce cas.

3.2.4 Benchmark

Le benchmark obtenu met en évidence les résultats suivants:

```
=== BENCHMARK MULTIPLICATION (Toeplitz & Random) ===
Size (128x128) X (128x128) :
  Operation       | Average (ms) | Median (ms)
  ----------------|--------------|--------------
  Toe-Toe (Mine)  |  6.289e-01   |  5.995e-01
  Toe-Toe (Flint) |  3.032e+00   |  2.936e+00
  Toe-Vec (Mine)  |  1.221e-01   |  1.197e-01
  Toe-Vec (Flint) |  3.217e-02   |  2.648e-02
Size (1024x1024) X (1024x1024) :
  Operation       | Average (ms) | Median (ms)
  ----------------|--------------|--------------
  Toe-Toe (Mine)  |  3.736e+00   |  3.704e+00
  Toe-Toe (Flint) |  9.842e+02   |  9.823e+02
  Toe-Vec (Mine)  |  6.383e-01   |  6.412e-01
  Toe-Vec (Flint) |  1.554e+00   |  1.499e+00
Size (4096x4096) X (4096x4096) :
  Operation       | Average (ms) | Median (ms)
  ----------------|--------------|--------------
  Toe-Toe (Mine)  |  1.242e+01   |  1.531e+01
  Toe-Toe (Flint) |  3.981e+04   |  4.273e+04
  Toe-Vec (Mine)  |  2.063e+00   |  2.532e+00
  Toe-Vec (Flint) |  1.763e+01   |  2.104e+01
```

-Pour les petites matrices (128x128): notre fonction de multiplication avec les générateurs de déplacement est plus rapide que la fonction naive de flint (6.289e-01 ms vs 3.032 ms), par contre notre fonction de multiplication de vecteur avec les générateurs est plus lente que la fonction flint (1.221e-01 ms vs 3.217e-02) -Pour les matrices de tailles 1024x1024: notre fonction de multiplication avec les générateurs de déplacement reste plus rapide que la fonction naive de flint (3.736 ms vs 9.842e+02 ms) avec un écart 10 fois plus important, cette fois-ci notre fonction de multiplication de vecteur est plus rapide que celle de flint (6.383e-01 ms vs 1.554 ms) -Pour de grandes matrices (4096x4096): notre fonction de multiplication avec les générateurs de déplacement reste plus rapide que la fonction naive de flint (1.242e+01 ms vs 3.981e+04 ms) avec un écart 10 fois plus important, la encore, notre fonction de multiplication de vecteur demeure plus rapide que celle de flint (2.063 ms vs 1.763e+01 ms)

(ici courbe des temps) blablabla...explication complexité...blablabla

3.3 Generator Compression

Any operation done on Toeplitz matrices will increase the rank of the generators. To maintain the complexity advantage of this structure, we must perform compressions. This will appear more and more on algorithms that require arithmetics.

For any pair of matrices $G_A, H_A \in \mathbb{K}^{n\times k}$, let r be the rank of their product $G_A H_A^T$ (where $r \leq k$). Our goal is to construct a new pair generators $G_D, H_D \in \mathbb{K}^{n\times r}$ such these conditions are met:

1. The displacement product is preserved:$G_A H_A^T = G_D H_D^T$
2. The number of columns of the matrices G_D and H_D are equal and is the rank of $G_A H_A^T$.

We will achieve this by independently compression both matrices and applying the inverse of the operations done to the other.

Algorithm 1 Generator Compression

```
procedure GENERATOR_COMPRESS(G_A, H_A)
    G_1, H_1 ← GENERATOR_COMPRESS_AUX(G_A, H_A)
    H_D, G_D ← GENERATOR_COMPRESS_AUX(H_1, G_1)
    return G_D, H_D
end procedure
```

Algorithm 2 Generator Compression Auxilary

```
procedure GENERATOR_COMPRESS_AUX(G, H)
    P, L, U ← LU_Decomposition(G^T)          ▷ PG^T = LU
    r ← Number of non-zero rows in U          ▷ rank
    if r < 1 then
        return Generators of a 0 matrix
    end if
    G_D ← (U_[0:r−1],*)^T
    H_D ← HP^T × L_*,[0:r−1]
    return G_D, H_D
end procedure
```

Computing the LU factorization of the transposed generator $G^T \in \mathbb{K}^{r\times n}$ requires $\mathcal{O}(r^2 n)$ operations. Extracting U and applying the permutation matrix P^T to H is in $\mathcal{O}(rn)$. Computing the new right factor $H_D = HP^T \times L_{*,[0:r-1]}$ involves multiplying an $n \times k$ matrix by a $k \times r$ matrix. Using standard matrix multiplication, this takes $\mathcal{O}(r^2 n)$ operations. Our implementation is using loops to get the selected amount of rows and columns but their complexities are bounded by $\mathcal{O}(r^2 n)$.

We can generalize the complexity by revising for any matrix in $\mathbb{K}^{m\times n}$ the overall time complexity for the generator compression is $\mathcal{O}(rmn)$.

Our implementation can be found in:

```
Project
\   src
|   \   compression
|   |   |   compression.c
|   |   |   compression.h
```

int gr_mat_generator_compress(gr_mat_t G_d, gr_mat_t H_d, gr_ctx_t ctx); Is our function that implements the algorithms talked in this chapter.

3.4 Inversion

3.4.1 Introduction

In this section we will study how we implemented Strassen's inversion method on displacement generators. Algorithm follows:

Algorithm 4 Strassen-type Inversion for Displacement Generators (adapted from [1])

procedure StrassenInverseGenerators(G_A, H_A)
  if $n = 1$ **then** ▷ **Step 1: Base Case**
    return G_D H_D representing the inverse of the scalar value of A from G_A, H_A
  end if

  $G_a, H_a, \ldots, G_d, H_d \leftarrow$ SplitQuadrants(G_A, H_A) ▷ **Step 2: Split into Quadrants**

  $G_e, H_e \leftarrow$ StrassenInverseGenerators(G_a, H_a) ▷ $e := a^{-1}$ ▷ **Step 3: Recursion #1**

  $G_{ce}, H_{ce} \leftarrow$ Compress(Mul(G_c, H_c, G_e, H_e)) ▷ **Step 4: Schur Complement**
  $G_{eb}, H_{eb} \leftarrow$ Compress(Mul(G_e, H_e, G_b, H_b))
  $G_{ceb}, H_{ceb} \leftarrow$ Compress(Mul(G_c, H_c, G_{eb}, H_{eb}) $\times -1$)
  $G_S, H_S \leftarrow$ Compress(Add($G_d, H_d, G_{ceb}, H_{ceb}$)) ▷ $S := d - ceb$

  $G_t, H_t \leftarrow$ StrassenInverseGenerators(G_S, H_S) ▷ $t := S^{-1}$ ▷ **Step 5: Recursion #2**

  $G_{ebt}, H_{ebt} \leftarrow$ Compress(Mul(G_{eb}, H_{eb}, G_t, H_t)) ▷ **Step 6: Strassen Assembly**
  $G_{tce}, H_{tce} \leftarrow$ Compress(Mul(G_t, H_t, G_{ce}, H_{ce}))
  $G_{ebtce}, H_{ebtce} \leftarrow$ Compress(Mul($G_{ebt}, H_{ebt}, G_{ce}, H_{ce}$))

  $G_x, H_x \leftarrow$ Compress(Add($G_e, H_e, G_{ebtce}, H_{ebtce}$)) ▷ $x := e + ebtce$
  $G_y, H_y \leftarrow$ Negate(G_{ebt}, H_{ebt}) ▷ $y := -ebt$
  $G_z, H_z \leftarrow$ Negate(G_{tce}, H_{tce}) ▷ $z := -tce$

  $G_{Inv}, H_{Inv} \leftarrow$ PackQuadrants($G_x, H_x, G_y, H_y, G_z, H_z, G_t, H_t$) ▷ **Step 7: Pack**
  return Compress(G_{Inv}, H_{Inv})
end procedure

This algorithm is based on Algorithm 10.1 from [1, Algorithme 10.1]. Notice that we strictly operate on Φ_+ displacement and bypassing Φ_- generators.

Proof. We proceed by induction on the size of the matrix $n = 2k$. Let $\Phi_+(X) = X - ZXZ^T$ be the displacement operator.

Base Case ($n = 1$): For a scalar matrix $A = [a]$, the operator Z is a 1×1 zero matrix. The displacement of A is $\Phi_+(A) = A - 0 \cdot A \cdot 0 = A$.

Inductive Step: Consider a matrix A of size $2k \times 2k$ represented by Φ_+ generators. The Strassen-type inversion relies on splitting the sub-blocks a, b, c, d which are represented by Φ_+ displacements, recursive calls to invert top-left block a ($e = a^{-1}$) and antoher for the Schur complement $t = S^{-1}$ and each outputs a Φ_+ generator. Lastly, multiplication and addition algorithms are taking a generator of Φ_+ displacement and returns a generator of Φ_+ displacement. Each computation and the final assembly remain in Φ_+ displacement.

Conclusion: Because the base case returns the inverse to Φ_+ generators, and the arithmetics are on Φ_+ generator operations, by induction, we maintain Φ_+ displacement during the entire execution. □

3.4.2 Split and Pack quadrants from displacement generators

Any Strassen-type divide and conquer algorithm will divide the problem to create sub-problems to economise in complexity. This type of approach on displacement generators require arithmetic opera-

tions to ensure the data stability during execution.

Main concern regarding this approach for our implementation of Toeplitz matrices in this project is that the algorithm doesn't have direct access to the dense matrix A. Any splitting and packing operation done will be on the generators G_A and H_A which represent the matrix entirely by default. Challenge of applying this split and pack comes from the displacement operator Φ_+ affecting the dense matrix entirely.

Let a dense matrix A be split into four quadrants:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

The shift operator Z acts on the entire matrix resulting elements from blocks to cross its boundaries. Precisely the last column of a shifted down into b. Last row of a shifted down into c. Last row of b, last column of c and the bottom-right element of a shifted down into d.

When we attempt to evaluate the local displacement of a sub-block (for example, $\Phi_+(d)$), we cannot simply extract the rows from the global generators G_A and H_A. The global generators are corrupted by the boundary elements of a that shifted into b and c.

To correctly split or pack the generators without having access to the dense matrix A, we must mathematically isolate and correct these boundary crossings which we call bleeding.

Splitting: During the Split, our goal is to extract the generators for the sub-blocks a, b, c, d using only the global generators G_A and H_A.

We start by spliting literally the generators G_A and H_A:

$$G = \begin{pmatrix} G_{TOP} \\ G_{BOTTOM} \end{pmatrix}, \quad H = \begin{pmatrix} H_{TOP} \\ H_{BOTTOM} \end{pmatrix}$$

We define $n_1 = ceil(n/2)$ and $n_2 = floor(n/2)$. Let G_{TOP} and H_{TOP} be a matrix of size $n_1 \times rank$ and G_{BOTTOM} and H_{BOTTOm} be a matrix of size $n_2 \times rank$. This allows us to create a split dimension which handles odd and even numbers.

For a, there are no bleed corrections. This block is the one that bleeds to all other blocks, in other words, there are no external shifts into it. Thus, its generators are simply the truncated top halves of the global generators:

$$G_a = G_{TOP}, \quad H_a = H_{TOP}$$

For b, block a bleeds into it from the left. Specifically the last column of block a shifts rightwards across the boundary into the first column of block b. To correct this bleed, we need to define the vector $e^{(m)}$ that has the dimensions of $m \times 1$: Let $e_0^{(m)}$ be a column vector with a 1 at the top, and $e_{m-1}^{(m)}$ be a column vector with a 1 at the bottom:

$$e_0^{(m)} = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad e_{m-1}^{(m)} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}$$

We calculate v_a, the last column of a. This column must contain the data for the bleed values, meaning that we are forced to partially calculate that part of the dense matrix. We rely on Φ_+ displacement reconstruction formula.

For any cell at row i and column j, the value of the dense matrix A is given by the sum of the shifted generator products, bounded by the diagonal limit $\min(i, j)$:

$$A_{i,j} = \sum_{k=0}^{r-1} \sum_{x=0}^{\min(i,j)} G_A[i-x, k] H_A[j-x, k]$$

where r is the displacement rank. The first sum of k is to run the columns of the displacement generators to retrieve the base values. The second sum over x essentially climbs up diagonally through the displacement matrix since Φ_+ displacement substracts diagonally the shifted matrix, we must add these shifted generator products back together to reconstruct the true value and stop when we hit the matrix boundary at min(i,j).

Since a is an $n_1 \times n_1$ matrix, the global column index for v_a is $j = n_1 - 1$ (the left end of the block a). For any row index i within this block ($0 \leq i < n_1$), we are guaranteed that $i \leq n_1 - 1$. So the boundary condition simplifies to $\min(i, n_1 - 1) = i$. We can write the i-th element of the vector v_a as:

$$(v_a)_i = \sum_{k=0}^{r-1} \sum_{x=0}^{i} G_A[i-x, k] H_A[n_1 - 1 - x, k]$$

By doing this, we evade the total reconstruction of a matrix and do a vector extraction which has the complexity $\mathcal{O}(n_1 \cdot r)$.

We shift v_a down using Z_{n_1} and append it to G_{TOP}. The corresponding H generator receives the vector $e_0^{(n_2)}$ to activate this correction exclusively on the first column of b when multiplied:

$$G_b = \begin{pmatrix} G_{TOP} & Z_{n_1} v_a \end{pmatrix}, \quad H_b = \begin{pmatrix} H_{BOTTOM} & e_0^{(n_2)} \end{pmatrix}$$

For block c, data bleeds into it from above. The last row of block a shifts downwards across the boundary. Just like to block b, we calculate r_a, the last row of a.

$$(r_a)_j = \sum_{k=0}^{r-1} \sum_{x=0}^{j} G_A[n_1 - 1 - x, k] H_A[j-x, k]$$

Because H matrices represent the transposed components of the displacement (in this case row instead of column), we apply the shift Z_{n_1} to r_a and append it to H_{TOP}:

$$G_c = \begin{pmatrix} G_{BOTTOM} & e_0^{(n_2)} \end{pmatrix}, \quad H_c = \begin{pmatrix} H_{TOP} & Z_{n_1} r_a \end{pmatrix}$$

Block d has the most bleeding, as it receives shifts from the last column of c (v_c) shifting right, the last row of b (r_b) shifting down, and the bottom-right scalar of a (s_a) shifting diagonally into the top-left cell of d. We extract v_c, r_b, and s_a from the global generators continuing the logic we have applied to block a:

$$(v_c)_l = \sum_{k=0}^{r-1} \sum_{x=0}^{n_1-1} G_A[n_1 + l - x, k] H_A[n_1 - 1 - x, k]$$

$$(r_b)_m = \sum_{k=0}^{r-1} \sum_{x=0}^{n_1-1} G_A[n_1 - 1 - x, k] H_A[n_1 + m - x, k]$$

s_a is a single scalar value located at the exact bottom-right corner of block a. Its global row and column indices are both exactly $n_1 - 1$, the boundary limit is $\min(n_1 - 1, n_1 - 1) = n_1 - 1$.

$$s_a = \sum_{k=0}^{r-1} \sum_{x=0}^{n_1-1} G_A[n_1 - 1 - x, k] H_A[n_1 - 1 - x, k]$$

We scale the basis vector $e_0^{(n_2)}$ by s_a and add it to the shifted column vector $Z_{n_2} v_c$. We create a double correction structure appended to the original truncated generators:

$$G_d = \begin{pmatrix} G_{BOTTOM} & s_a e_0^{(n_2)} + Z_{n_2} v_c & e_0^{(n_2)} \end{pmatrix}, \quad H_d = \begin{pmatrix} H_{BOTTOM} & e_0^{(n_2)} & Z_{n_2} r_b \end{pmatrix}$$

Packing

Pack performs the exact inverse of the splitting operation. After computing the inverse sub-blocks (we note as x, y, z, t), our goal is to get the global generators G_{Inv} and H_{Inv} for the full inverse matrix A^{-1}.

If we were to concatenate the local generators, the result would assume that the boundaries between the blocks are filled with zeros. Because the operator Φ_+ shifts data across these boundaries, just like during split, we must re-inject the bleeding corrections to restore the data integrity.

Just as we calculated boundaries, we partially reconstruct the boundary vectors of the local inverse blocks using again the formula. Because we are now operating on isolated blocks' generators (rather than the single global array),the coordinates for each reconstruction does not need an offset (like $+n_1$ we did for split). So the diagonal shift boundries must be bounded by local dimensions.

Let r_x, r_y, and r_z be the respective displacement ranks of these blocks, and let w represent the diagonal shift index. Block x is an $n_1 \times n_1$ matrix. Because it is a square block, its boundaries mirror the exact logic we used for block a. The last column (v_x), last row (r_x), and bottom-right scalar (s_x) are computed strictly from G_x and H_x:

$$(v_x)_i = \sum_{k=0}^{r_x-1} \sum_{w=0}^{i} G_x[i-w, k] H_x[n_1 - 1 - w, k] \quad \text{for } 0 \le i < n_1$$

$$(r_x)_j = \sum_{k=0}^{r_x-1} \sum_{w=0}^{j} G_x[n_1 - 1 - w, k] H_x[j - w, k] \quad \text{for } 0 \le j < n_1$$

$$s_x = \sum_{k=0}^{r_x-1} \sum_{w=0}^{n_1-1} G_x[n_1 - 1 - w, k] H_x[n_1 - 1 - w, k]$$

Block z has dimensions $n_2 \times n_1$. The last column is fixed at local index $j = n_1 - 1$, while the row index l varies up to $n_2 - 1$. Unlike split, l is no longer offset by a global position. Therefore, we must do the diagonal shift using $\min(l, n_1 - 1)$ to ensure the reconstruction does not exceed the dimensions of z:

$$(v_z)_l = \sum_{k=0}^{r_z-1} \sum_{w=0}^{\min(l,\ n_1-1)} G_z[l - w, k] H_z[n_1 - 1 - w, k] \quad \text{for } 0 \le l < n_2$$

Same for the block y which has the dimensions $n_1 \times n_2$. The last row is fixed at index $i = n_1 - 1$, and the column index m varies up to $n_2 - 1$. Again, because we are operating within local coordinates, the diagonal shift is bounded by $\min(n_1 - 1, m)$:

$$(r_y)_m = \sum_{k=0}^{r_y-1} \sum_{w=0}^{\min(n_1-1,\ m)} G_y[n_1 - 1 - w, k] H_y[m - w, k] \quad \text{for } 0 \le m < n_2$$

To apply these corrections, we construct a large set of global generators. We horizontally concatenate the local generators of x, y, z, t into their respective quadrants. To correct the displacement boundaries, we append four specific correction columns (c_1, c_2, c_3, c_4).

We define these giant global generators as block matrices:

$$G_{RES} = \begin{pmatrix} G_x & G_y & 0 & 0 & -Z_{n_1} v_x & 0 & 0 & 0 \\ 0 & 0 & G_z & G_t & 0 & -e_0^{(n_2)} & -(s_x e_0^{(n_2)} + Z_{n_2} v_z) & -e_0^{(n_2)} \end{pmatrix}$$

$$H_{RES} = \begin{pmatrix} H_x & 0 & H_z & 0 & 0 & Z_{n_1} r_x & 0 & 0 \\ 0 & H_y & 0 & H_t & e_0^{(n_2)} & 0 & e_0^{(n_2)} & Z_{n_2} r_y \end{pmatrix}$$

Notice that the boundary correction columns in G_{RES} has negative signs. The displacement operation subtracts the diagonally shifted matrix: $\Phi_+(A^{-1}) = A^{-1} - ZA^{-1}Z^T$. When the local inverse blocks (x, y, z, t) are computed, their local displacements (ex. $G_y H_y^T$) are evaluated as if the blocks exist alone, assuming that any data shifted outside their local boundaries is replaced by zeros.

Within the global structure of A^{-1}, these blocks are next to eachother. The global shift operator Z pushes data across these block boundaries. Any data that bleeds from block x into block y must physically enter the local equation of y as a subtracted value. By negating bleeds on the appended correction columns in G_{RES}, the generator multiplication $G_{RES} H_{RES}^T$ produces these required negative terms.

To prove the mathematical correctness of these correction columns, we can explicitly compute the global displacement reconstruction by multiplying the uncompressed generators: $\Phi_+(A^{-1}) = G_{RES} H_{RES}^T$.

$$G_{RES} H_{RES}^T = \begin{pmatrix} G_x H_x^T & G_y H_y^T - Z_{n_1} v_x (e_0^{(n_2)})^T \\ G_z H_z^T - e_0^{(n_2)} (Z_{n_1} r_x)^T & G_t H_t^T - Z_{n_2} v_z (e_0^{(n_2)})^T - e_0^{(n_2)} (Z_{n_2} r_y)^T - s_x e_0^{(n_2)} (e_0^{(n_2)})^T \end{pmatrix}$$

The terms ($G_x H_x^T$, $G_y H_y^T$, etc.) represent the isolated local displacements of the sub-matrices. The subtracted terms are the bleeds. For example the term $-Z_{n_1} v_x (e_0^{(n_2)})^T$ subtracts the shifted last column of block x into the first column of block y.

This implementation exposes a vulnerability: Let r_x, r_y, r_z, r_t be the displacement ranks of the quadrants. The rank of our assembled matrix is:

$$\text{Rank}_{RES} = r_x + r_y + r_z + r_t + 4$$

Passing this uncompressed matrix up to the next recursion level would cause the rank to explose. Therefore, immediately after constructing G_{RES} and H_{RES}, we compress them.

3.4.3 Conclusion

The advantage of this inversion lies on displacement generators rather than dense matrix representations like any other algorithm mentioned in this report.

By operating on generators of size $n \times r$ (where the displacement rank $r \ll n$), the complexity in memory is reduced from the standard $\mathcal{O}(n^2)$ down to $\mathcal{O}(r \cdot n)$. Assuming optimal generator multiplication, the complexity of the recursive Strassen-type assembly falls significantly below the $\mathcal{O}(n^3)$ or $\mathcal{O}(n^{2.81})$ operations required by classical dense matrix inversion.

Despite these gains, the structured approach introduces overhead due to strict displacement boundary management. The split and pack require continuous $\mathcal{O}(r \cdot n)$ diagonal vector extractions to mathematically correct the Φ_+ bleeding. More critically, the structural reassembly during the pack and each multiplication and addition increases the rank, requiring more computation for compressions at every recursive return.

While highly optimal for large matrices with a strictly bounded rank, this heavy structural overhead makes the algorithm computationally disadvantageous for small matrices compared to standard dense inversion.

```
mention a threshold
mention and show benchmarks once sure and verified
```

As detailed by Victor Y. Pan in *Structured Matrices and Polynomials* [2, Chapter 5], the alternative is the Morf-Bitmead-Anderson (MBA) algorithm. Originally proposed by Morf (1980) and Bitmead and Anderson (1980), achieving $\mathcal{O}(n \log^2 n)$ complexity for Toeplitz-like and Hankel-like matrices. However, because it is also a divide-and-conquer method, the MBA approach still requires calculations of quadrants and generator compression.

To bypass the need to split the matrix, one should prefer a Newton iterative inversion. Pan dedicates significant analysis to this approach in [2, Chapters 6 and 7], (detailing based on the Schulz iteration: $X_{k+1} = X_k(2I - AX_k)$).

```
conclude even more
```

References

[1] Alin Bostan, Frédéric Chyzak, Marc Giusti, Romain Lebreton, Grégoire Lecerf, Bruno Salvy, and Éric Schost. Algorithmes efficaces en calcul formel, August 2017. 686 pages. Édition 1.0.

[2] Victor Y. Pan. *Structured Matrices and Polynomials: Unified Superfast Algorithms*. Birkhäuser & Springer, 2001.